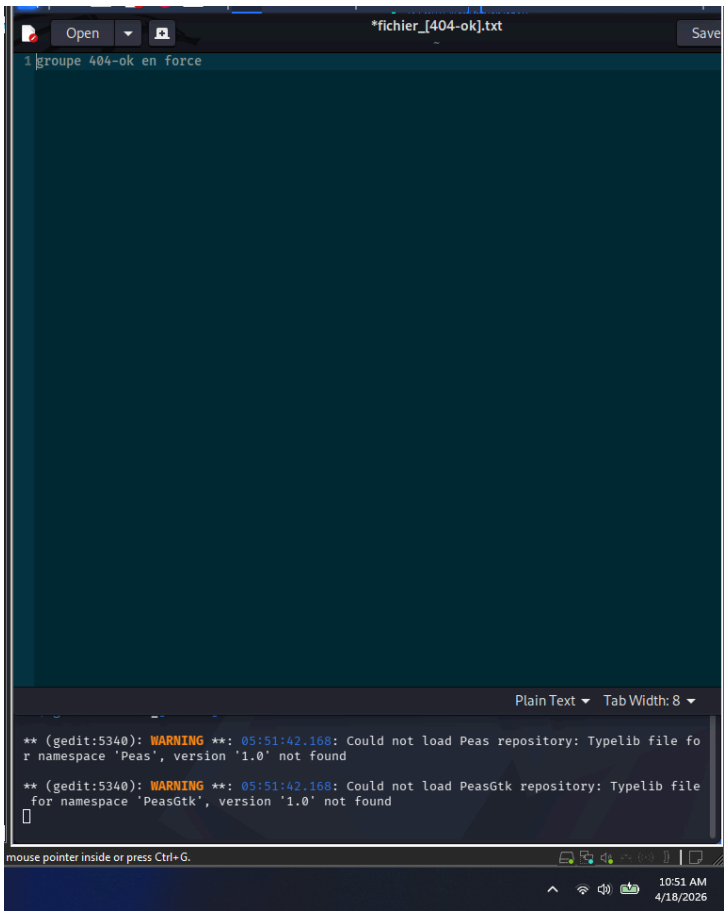
```
┌──(kali㉿kali)-[~]
└─$ gedit fichier_[404-ok].txt

** (gedit:4244): WARNING **: 05:50:33.797: Could not load Peas repository: Typelib file fo
r namespace 'Peas', version '1.0' not found

** (gedit:4244): WARNING **: 05:50:33.797: Could not load PeasGtk repository: Typelib file
 for namespace 'PeasGtk', version '1.0' not found
```

mouse pointer inside or press Ctrl+G.

10:51 AM
4/18/2026



Dans cette capture, nous observons **la toute première étape fondamentale** de ce TP, à savoir la création du fichier texte en clair (le fichier initial).

Ici, on remarque que vous avez utilisé la commande gedit fichier_[404-ok].txt dans le terminal pour ouvrir l'éditeur de texte graphique. Dans cet éditeur, vous avez saisi la phrase **"groupe 404-ok en force"** avant d'enregistrer le fichier. C'est ce fichier précis qui sert de point de départ pour toutes les opérations ultérieures, telles que le chiffrement symétrique (avec les algorithmes RC4 et DES). C'est également ce document (une fois chiffré) qui est censé être transmis à votre "camarade" pour qu'il procède au déchiffrement, comme l'exige le fascicule.

```
ile.c:75:

┌──(kali㉿kali)-[~]
└─$ openssl enc -rc4 -in fichier_[404-ok].txt -out fichier_[404-ok].enc
enter RC4 encryption password:
Verifying - enter RC4 encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

```
┌──(kali㉿kali)-[~]
└─$ cat fichier_[404-ok].enc
Salted__◆◆◆oQ6◆.◆zT◆◆◆◆◆Q◆kv◆EkZS-◆◆5s

┌──(kali㉿kali)-[~]
└─$
```

```
nouse pointer inside or press Ctrl+G.
```

10:53 AM
4/18/2026

Dans cette capture, nous passons à la phase de **chiffrement symétrique** en utilisant l'algorithme RC4.

La première partie de l'image montre l'exécution de la commande openssl enc -rc4 -in fichier_[404-ok].txt -out fichier_[404-ok].enc. Lors de cette étape, OpenSSL a demandé la saisie d'un mot de passe de chiffrement afin d'en dériver une clé sécurisée, conformément aux indications du fascicule.

La deuxième partie de la capture illustre l'étape de vérification. En exécutant la commande cat fichier_[404-ok].enc, on constate que le contenu du fichier chiffré généré s'affiche sous forme de caractères spéciaux et illisibles (commençant par "Salted__"). Cela confirme avec succès que le texte clair a bien été transformé et que le message est désormais totalement inintelligible, comme exigé par les instructions du TP.

```
┌──(kali㉿kali)-[~]
└─$ openssl enc -rc4 -in fichier_[esm_mte3ek].txt -out fichier_[esm_mte3ek].enc
Can't open "fichier_[esm_mte3ek].txt" for reading, No such file or directory
40C761EA227F0000:error:80000002:system library:BIO_new_file:No such file or directory:../c
rypto/bio/bss_file.c:67:calling fopen(fichier_[esm_mte3ek].txt, rb)
40C761EA227F0000:error:10000080:BIO routines:BIO_new_file:no such file:../crypto/bio/bss_f
ile.c:75:

┌──(kali㉿kali)-[~]
└─$ openssl enc -rc4 -in fichier_[404-ok].txt -out fichier_[404-ok].enc
enter RC4 encryption password:
Verifying - enter RC4 encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(kali㉿kali)-[~]
└─$ cat fichier_[404-OK].enc
cat: 'fichier_[404-OK].enc': No such file or directory

┌──(kali㉿kali)-[~]
└─$ cat fichier_[404-ok].enc
Salted__◆◆◆oQ6◆.◆zT◆◆◆◆◆Q◆kv◆EkZS-◆◆5s

┌──(kali㉿kali)-[~]
└─$ openssl enc -rc4 -d -in fichier_[404-ok].enc -out fichier_[404-ok]_dechiff.txt
enter RC4 decryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(kali㉿kali)-[~]
└─$ diff fichier_[404-ok].txt fichier_[404-ok]_dechiff.txt

┌──(kali㉿kali)-[~]
└─$ openssl enc -des -in fichier_[404-ok].txt -out fichier_chiff_des.enc -k 0123456789ABCD
EF
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(kali㉿kali)-[~]
└─$ openssl enc -des -d -in fichier_chiff_des.enc -out fichier_dechiff_des.txt -k 01234567
89ABCDEF
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(kali㉿kali)-[~]
└─$ openssl enc -des -d -in fichier_chiff_des.enc -out fichier_dechiff_des.txt -k 01234567
89ABCDEF
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

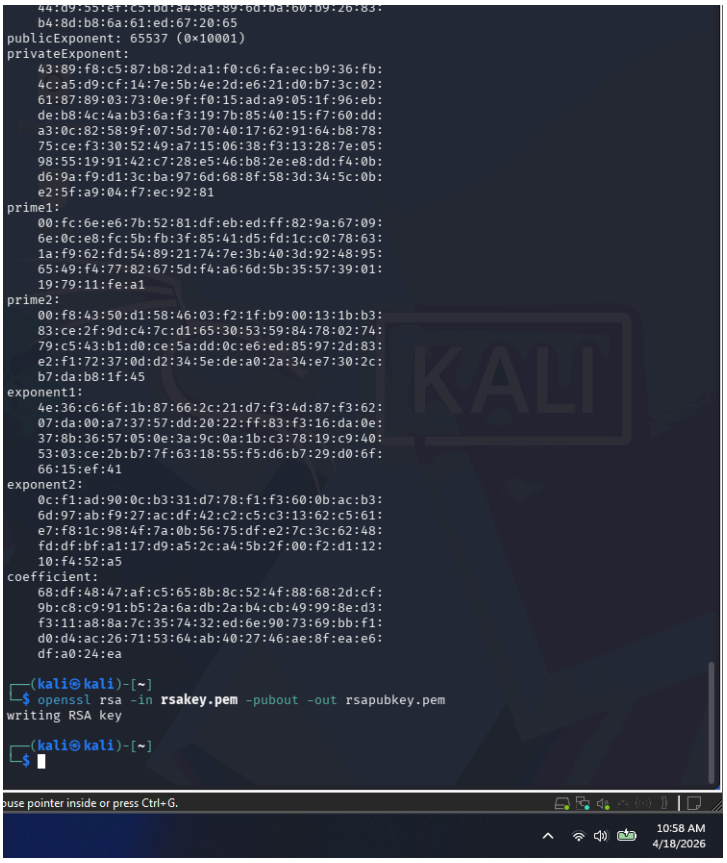
ouse pointer inside or press Ctrl+G.

10:57 AM
4/18/2026

```
┌──(kali㉿kali)-[~]
└─$ diff fichier_[404-ok].txt fichier_dechiff_des.txt

┌──(kali㉿kali)-[~]
└─$ openssl genrsa -out rsakey.pem 1024

┌──(kali㉿kali)-[~]
└─$ cat rsakey.pem
-----BEGIN PRIVATE KEY-----
MIICdQIBADANBgkqhkiG9w0BAQEFAASCAl8wggJbAgEAAoGBAPTNz/7f0EbSkja7
.hcEftrg5nUCnIFcFv9xmxCPwpLoUgxgIyfPQs912jSV2JPftGEfjesF8PIa3Jw8
0AEm21smrgxbwM5PXUsz5567CrO4H9DeSCqDfhA3o9v5WtD432GxnAt74BRRNlV
8W9pI6JbbpguSaDtI24amHtZyBlAgMBAAECgYBDifjFh7gtofDG+uy5NvtMpdnP
H5bTi3mIdC3PAJhh4kDcw6f8BWtqQUfluveuExKs2rzGXuFQBX3YN2jDIJYnwdd
EAXYpFkuHh1zvMwUkmnFQY48xMofgWYVRmRQsco5Ua4Lujd9AvWmvnRPLqXbWiP
D00XAviX6kE9+ySgQJBAPxu5ntSgd/r7f+CmmcJbgzo/Fv7P4VB1f0cwHhjGvli
VSJIXR+O0A9kkiVZUn0d4JnXfSmbVs1VzkBGXkR/qECQQD4Q1DRWEYD8h+5ABMb
4POL53EfNFlMFNZhHgCdHnFQ7HQzlrdDObthZctg+LxcjcN0jRe3qAqNOcwLLfa
B9FAkBONsZvG4dmLCHX802H82IH2gCnN1fdICL/g/MW2g43izZXBQ46nAobw3gZ
UBTA84rt39jGFX11rcp0G9mFe9BAkAM8a2QDLMx13jx82ALrLNtl6v5J6zfQsLF
xNixWHn+ByYT3oLVnXf4nw8Ykj937+hF9mlLKRbLwDy0RIQ9FKlAkBo30hHr8Vl
4xST4hoLc+byMmRtSpq2yq0y0mZjtPzEaiKfDV0Mu1ukHNpu/HQ1KwmcVNkq0An
Rq6P6ubfoCTq
-----END PRIVATE KEY-----

┌──(kali㉿kali)-[~]
└─$ openssl rsa -in rsakey.pem -text -noout
Private-Key: (1024 bit, 2 primes)
modulus:
    00:f4:cd:cf:fe:df:d0:46:d2:92:36:bb:8a:17:04:
    7e:da:e0:e6:75:02:9c:81:5c:16:ff:71:9b:10:8f:
    c2:92:e8:52:0c:60:23:27:cf:42:cf:75:da:34:95:
    d8:93:df:b4:61:1f:8d:eb:05:f0:f2:1a:dc:9c:3c:
    9c:e0:04:9b:6d:6c:9a:b8:31:6f:03:39:3d:75:2c:
    cf:9e:7a:ec:2a:ce:e0:7f:43:79:20:aa:0d:f8:40:
    de:8f:6f:e5:6b:43:e3:7d:86:c6:70:2d:ef:80:51:
    44:d9:55:ef:c5:bd:a4:8e:89:6d:ba:60:b9:26:83:
    b4:8d:b8:6a:61:ed:67:20:65
publicExponent: 65537 (0x10001)
privateExponent:
    43:89:f8:c5:87:b8:2d:a1:f0:c6:fa:ec:b9:36:fb:
    4c:a5:d9:cf:14:7e:5b:4e:2d:e6:21:d0:b7:3c:02:
    61:87:89:03:73:0e:9f:f0:15:ad:a9:05:1f:96:eb:
    de:b8:4c:4a:b3:6a:f3:19:7b:85:40:15:f7:60:dd:
    a3:0c:82:58:9f:07:5d:70:40:17:62:91:64:b8:78:
    75:ce:f3:30:52:49:a7:15:06:38:f3:13:28:7e:05:
    98:55:19:91:42:c7:28:e5:46:b8:2e:e8:dd:f4:0b:
```

se pointer inside or press Ctrl+G.

10:58 AM
4/18/2026



Ces captures mettent en évidence la réalisation des opérations de **chiffrement symétrique et asymétrique**. Dans un premier temps, le chiffrement symétrique est appliqué en utilisant successivement les algorithmes RC4 et DES. Après avoir chiffré puis déchiffré les fichiers, l'utilisation de la commande diff confirme que le contenu du fichier déchiffré est parfaitement identique à celui du fichier clair d'origine, validant ainsi la réversibilité et l'intégrité du processus.

Dans un second temps, le travail aborde la cryptographie asymétrique avec la génération d'une paire de clés RSA de 1024 bits via la commande openssl genrsa, sauvegardée dans le fichier rsakey.pem. L'inspection approfondie de cette clé en format texte (-text) permet d'observer ses composantes mathématiques détaillées en hexadécimal, telles que le *modulus*, l'exposant public et les nombres premiers (prime1 et prime2). Enfin, la dernière opération illustrée montre l'extraction réussie de la clé publique vers un nouveau fichier nommé rsapubkey.pem en utilisant l'option -pubout.

```
┌──(kali㉿kali)-[~]
└─$ openssl rsa -in rsakey.pem -pubout -out rsapubkey.pem
writing RSA key

┌──(kali㉿kali)-[~]
└─$ openssl enc -aes256 -in rsakey.pem -out rsakeyencaes.pem
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.

┌──(kali㉿kali)-[~]
└─$ openssl pkeyutl -pubin -inkey rsapubkey.pem -in fichier_[404-ok].txt -encrypt -out fic
hier_[404-ok]_rsa.enc

┌──(kali㉿kali)-[~]
└─$ openssl pkeyutl -inkey rsakey.pem -in fichier_[404-ok]_rsa.enc -decrypt -out fichier_[
404-ok]_rsa_dechiff.txt

┌──(kali㉿kali)-[~]
└─$ diff fichier_[404-ok].txt fichier_[404-ok]_rsa_dechiff.txt

┌──(kali㉿kali)-[~]
└─$ openssl dgst -md5 -out fichier_[404-ok].md5 fichier_[404-ok].txt

┌──(kali㉿kali)-[~]
└─$ openssl dgst -sha1 -out fichier_[404-ok].sha1 fichier_[404-ok].txt

┌──(kali㉿kali)-[~]
└─$
```

Dans cette capture, nous abordons la sécurisation des clés, le chiffrement asymétrique des données et l'introduction aux fonctions de hachage.

Dans un premier temps, la clé privée RSA générée précédemment a été sécurisée par un chiffrement symétrique utilisant l'algorithme AES-256. Ensuite, le mécanisme de chiffrement asymétrique a été testé de bout en bout sur le fichier texte : celui-ci a été chiffré à l'aide de la clé publique (rsapubkey.pem), puis déchiffré avec succès en utilisant la clé privée (rsakey.pem) via la commande pkeyutl. L'exécution de la commande diff, qui ne renvoie aucune sortie, prouve que le fichier déchiffré est rigoureusement identique au fichier clair d'origine.

Enfin, la dernière partie de la capture illustre le passage à la section de la signature numérique avec la génération d'empreintes. Les algorithmes de hachage cryptographique **MD5** et **SHA1** ont été appliqués au fichier initial, permettant de calculer ses condensats uniques qui ont été sauvegardés dans des fichiers portant les extensions .md5 et .sha1.

```
┌──(kali㉿kali)-[~]
└─$ openssl rsautl -verify -in fichier_sig -out empreinte2 -pubin -inkey rsapubkey.pem
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.

┌──(kali㉿kali)-[~]
└─$ openssl genrsa -out serveur_cle.pem 1024

┌──(kali㉿kali)-[~]
└─$ openssl req -new -key serveur_cle.pem -out serveur_cert.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:tunisia
String too long, must be at most 2 bytes long
Country Name (2 letter code) [AU]:TU
State or Province Name (full name) [Some-State]:ariana
Locality Name (eg, city) []:jabbes
Organization Name (eg, company) [Internet Widgits Pty Ltd]:esprit
Organizational Unit Name (eg, section) []:esprit
Common Name (e.g. server FQDN or YOUR name) []:sandid
Email Address []:sandidraslen1@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:mechwi3alouch
An optional company name []:

┌──(kali㉿kali)-[~]
└─$ openssl req -new -x509 -days 365 -key serveur_cle.pem -out serveur_cert.crt
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:tu
State or Province Name (full name) [Some-State]:ariana
Locality Name (eg, city) []:jabbes
Organization Name (eg, company) [Internet Widgits Pty Ltd]:esprit
Organizational Unit Name (eg, section) []:esprit
Common Name (e.g. server FQDN or YOUR name) []:sandid
Email Address []:sandidraslen1@gmail.com

┌──(kali㉿kali)-[~]
└─$
```

mouse pointer inside or press Ctrl+G.

11:05 AM
4/18/2026

```
┌──(kali㉿kali)-[~]
└─$ openssl x509 -in serveur_cert.crt -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            02:9a:dc:c6:9a:f2:56:70:9f:6c:e5:0c:c5:2a:7a:d3:7f:5b:79:97
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C=tu, ST=ariana, L=jabbes, O=esprit, OU=esprit, CN=sandid, emailAddress=sa
ndidraslen1@gmail.com
        Validity
            Not Before: Apr 18 10:05:32 2026 GMT
            Not After : Apr 18 10:05:32 2027 GMT
        Subject: C=tu, ST=ariana, L=jabbes, O=esprit, OU=esprit, CN=sandid, emailAddress=s
andidraslen1@gmail.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (1024 bit)
                Modulus:
                    00:c9:86:4f:b3:10:e5:bc:14:5b:44:d0:6f:55:fb:
                    05:a9:59:ce:e2:9b:50:31:65:6a:ca:80:86:a4:c9:
                    28:d5:df:9e:d6:81:d5:fb:a3:74:17:9d:bc:10:df:
                    25:3c:e6:dc:a3:28:1a:33:7b:9a:e6:2a:d1:cb:84:
                    e7:2d:36:0a:ea:29:02:1a:e2:c6:8b:22:40:41:48:
                    91:02:68:9b:13:c5:19:e1:c4:7b:b7:fb:b7:0e:0d:
                    b6:e2:c2:cc:64:ca:81:a0:25:2c:62:d2:27:1c:76:
                    3c:17:97:17:a6:b3:15:cc:6c:31:bc:e9:49:0a:f8:
                    1f:56:90:c6:87:ba:c6:4c:9d
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                90:BA:AA:64:5B:AA:16:93:87:3B:75:90:84:59:AD:DE:1F:29:53:03
            X509v3 Authority Key Identifier:
                90:BA:AA:64:5B:AA:16:93:87:3B:75:90:84:59:AD:DE:1F:29:53:03
            X509v3 Basic Constraints: critical
                CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        36:6c:41:18:61:30:53:da:e4:b0:a7:58:21:35:51:c4:dd:aa:
        1f:92:6c:9d:ae:98:b2:31:a6:f2:e5:de:56:73:62:63:c1:ad:
        d0:a0:1b:54:57:fa:9b:9a:95:47:f1:d4:90:1c:c9:c2:4a:19:
        7d:b3:e6:b7:99:c3:6b:02:2e:c7:7c:40:84:19:63:da:ba:93:
        39:4f:6d:ae:c9:78:72:36:02:fa:6e:39:96:36:6a:42:b4:bd:
        29:bb:6d:4b:a8:46:90:58:4f:c7:ad:38:82:10:bf:5c:51:e2:
        86:8a:a8:37:71:17:92:de:40:22:3b:40:a4:c0:e5:3b:0b:1e:
        64:9f

┌──(kali㉿kali)-[~]
└─$
```

mouse pointer inside or press Ctrl+G.

11:07 AM
4/18/2026

Ces captures démontrent le processus complet de **création et de vérification d'une signature numérique**.

Tout d'abord, l'empreinte de hachage (le fichier .sha1) générée à l'étape précédente a été signée numériquement à l'aide de la clé privée RSA (rsakey.pem), produisant ainsi un fichier de

signature nommé fichier_sig via l'option -sign. Cette étape est cruciale car l'utilisation de la clé privée garantit l'authenticité de l'expéditeur et la non-répudiation du message.

Ensuite, la deuxième capture illustre la phase de vérification. L'option -verify a été utilisée conjointement avec la clé publique (rsapubkey.pem) pour vérifier la signature et extraire le condensat vers un nouveau fichier nommé empreinte2. Enfin, l'exécution de la commande diff entre l'empreinte originale et l'empreinte extraite ne renvoie aucune sortie. Cela confirme que les deux empreintes sont rigoureusement identiques, prouvant de manière irréfutable que le fichier n'a subi aucune altération et que la signature est parfaitement valide.

Cette dernière capture illustre l'ultime étape du TP, qui consiste à simuler le rôle d'une **Autorité de Certification (AC)** pour valider et signer un certificat.

Dans la première partie de l'image, on observe la création d'un certificat racine auto-signé pour l'AC (ca.crt), d'une validité de 730 jours, généré à partir de sa clé privée (cakey.pem) via la commande openssl req. Lors de cette étape, les diverses informations d'identification de l'autorité (Pays : TU, État : ariana, Organisation : esprit, etc.) ont été correctement renseignées.

La seconde partie de la capture met en évidence l'aboutissement du processus de certification. La demande de certificat du serveur (le fichier .csr créé précédemment) est officiellement signée par notre propre Autorité de Certification en utilisant la commande openssl x509. L'apparition du message Signature ok, suivie de l'affichage des informations du sujet (notamment le nom de domaine www.groupe404.tn), confirme que le certificat final du serveur (serveur_cert_final.crt) a été généré avec succès et qu'il est désormais validé de manière cryptographique par l'AC.

Conclusion :

En guise de conclusion, ce travail pratique nous a permis d'explorer en profondeur les concepts fondamentaux de la cryptographie et de les appliquer concrètement grâce à l'outil OpenSSL sous un environnement Linux. Tout au long de ce TP, nous avons pu mettre en évidence les principes du chiffrement symétrique (avec les algorithmes RC4 et DES) garantissant la confidentialité des données, ainsi que ceux du chiffrement asymétrique (RSA), indispensable pour l'échange sécurisé et la protection des clés.

De plus, l'utilisation des fonctions de hachage (MD5 et SHA1) nous a permis de comprendre les mécanismes assurant l'intégrité des fichiers. En combinant ces différentes techniques, nous avons réussi à générer et à vérifier des signatures numériques, garantissant ainsi l'authenticité et la non-répudiation des messages. Enfin, la création de certificats numériques et la simulation d'une Autorité de Certification (AC) nous ont offert une vision globale et pratique sur la manière dont la confiance et la sécurité sont établies sur les réseaux modernes.

En somme, ce TP fut une expérience très enrichissante qui a permis de consolider nos connaissances théoriques en sécurité informatique par une pratique technique indispensable.